

React con TypeScript

N A N C Y B E R N A L



¿QUÉ ES TYPESCRIPT?

Es un lenguaje de programación de código abierto y muy completo.

Es desarrollado y mantenido por Microsoft.

También se le conoce como un superconjunto tipado para JavaScript, lo que significa que cualquier código de JavaScript válido es también código de TypeScript válido.

La principal característica de TypeScript es que agrega un sistema de tipos estático a JavaScript, lo que permite detectar errores y proporcionar herramientas de desarrollo más sólidas.



VENTAJAS DE TYPESCRIPT

- ✓ El sistema de tipado de tipos de TypeScript permite al programador especificar los tipos de variable, parámetros de función, valores de retorno y más.
- ✓ Capacidad de realizar la comprobación de tipos durante la compilación cuando estás escribiendo tu código y detectar posibles errores antes de que el código sea ejecutado.
- ✓ No necesitas que el proyecto esté en producción para revisar que todo esté bien.
- ✓ Con TypeScript puedes tener la garantía de que tu código va a funcionar correctamente.



PASANDO PROYECTO A TS

```
o Project name:
| audifonos-ts
|
o Select a framework:
| React
|
o Select a variant:
| TypeScript + SWC
|
o Scaffolding project in C:\Users\Nancy\Documents\UTFSM\2025\misEjemplos\audifonos-ts...
|
- Done. Now run:

cd audifonos-ts
npm install
npm run dev

PS C:\Users\Nancy\Documents\UTFSM\2025\misEjemplos> |
```

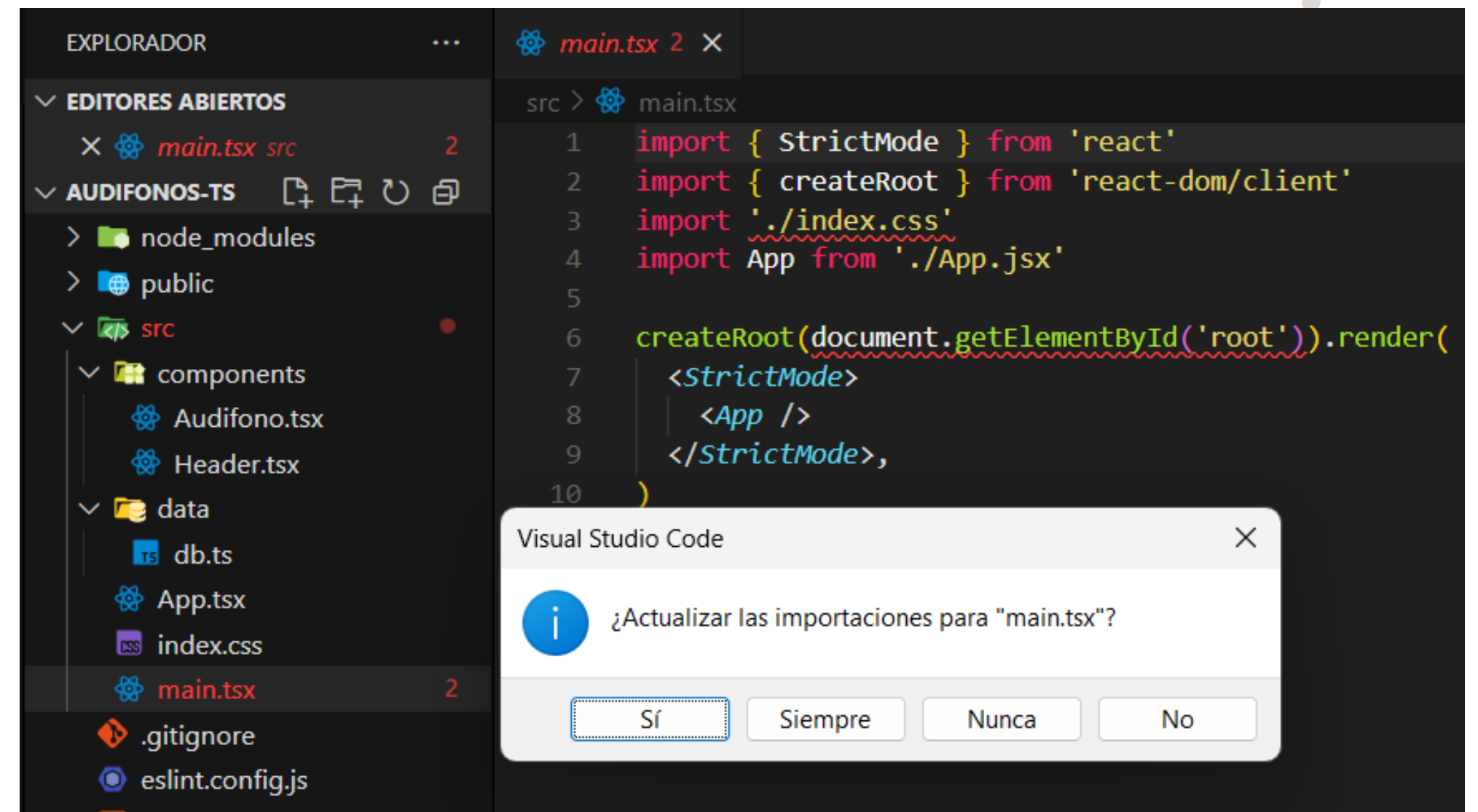
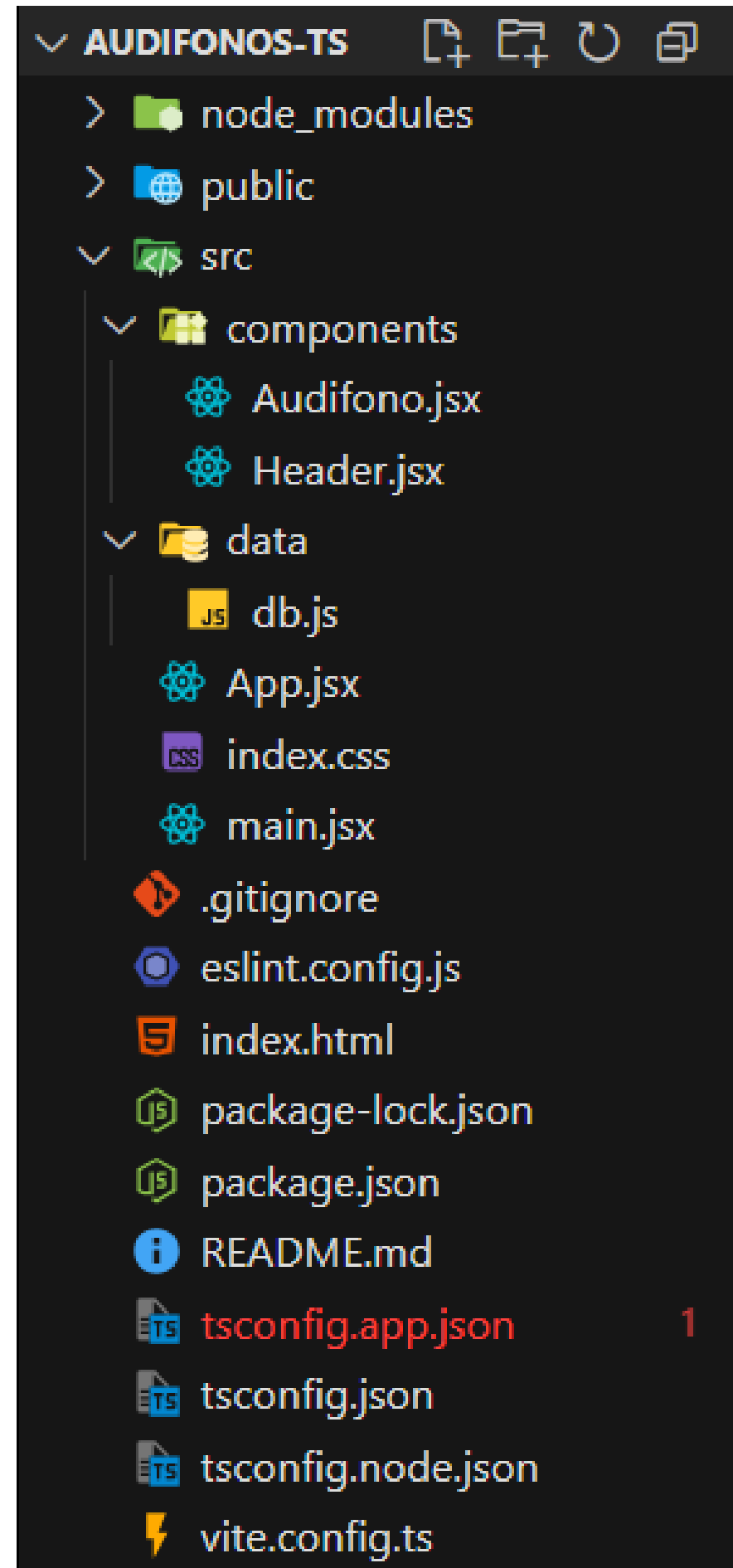
▼ AUDIFONOS-TS

- > node_modules
- > public
- ▼ src
 - > assets
 - App.css
 - App.tsx
 - index.css
 - main.tsx
 - vite-env.d.ts
 - .gitignore
 - eslint.config.js
 - index.html
 - package-lock.json
 - package.json
 - README.md
 - tsconfig.app.json
 - tsconfig.json
 - tsconfig.node.json
 - vite.config.ts



DE REACT-JS A REACT-TS

- **Abro mi proyecto existente y copio la carpeta img en public.**
- **Borro la carpeta src y copio la del proyecto anterior.**
- **Debo cambiar el nombre de los archivos .jsx a .tsx**





TS y NULL

- El error en el main no es un problema de versiones, no es un problema de escribir código un poco más moderno.
 - Lo que pasa aquí es que se hace referencia a un elemento que ts cree podría ser 'null'.
 - TypeScript utiliza algo que se le conoce como inferir el tipo de dato.
- Es por eso que aquí nos marca un error, porque estamos utilizando un parámetro que puede ser null y cuando hay algo que pueda ser null en TypeScript tienes que escribir código que garantice que no va a ser null o que revise en caso de que no sea null.

```
createRoot(document.getElementById('root')!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)  
  
// ! : Assertion not null  
// ó  
// non null assertion
```



TS: PRIMITIVE TYPES

Si revisamos, la página se ve correctamente, pero el código está marcando errores. Entonces hay que corregir algunas cosas, iniciando con los Primitive Types.

Los Primitive Types son los tipos de datos que soporta TypeScript de forma nativa.

Estos son los siguientes:

- number, String, boolean, null y undefined.
- Cuando creas un objeto, cada una de las propiedades, tienes que especificarle qué tipo de dato de estos primitive types es.
- Si creas una variable o una función también y si la función tiene parámetros, también hay que especificarle el tipo de Primitive type.



TS: TYPES & INTERFACE

La idea es evitar que ts infiera el tipo de dato y que uno como desarrollador lo defina.

Par eso se puede trabajar con Types & Interfaces.

En TypeScript, tanto type como interface se utilizan para definir la estructura de los datos, pero tienen algunas diferencias clave: type permite crear alias de tipos, uniones, intersecciones y tipos de mapeo, mientras que interface define la estructura de objetos y se puede extender con extends.

Es una forma de crear una estructura y definir qué tipo de dato tiene cada propiedad de un objeto o agrupar propiedades de un objeto.



TS: TYPES & INTERFACE

src > data > TS db.ts > [db]

```
1  type Audifono = {
2      id: number
3      name: string
4      image: string
5      description: string
6      price: number
7  }
8
9  export const db : Audifono[] = [
10     {
11         id: 1,
12         name: 'Pro X2',
13         image: 'audifono01',
14         description: 'Lorem ipsum, dolor sit
15         price: 199,
16     },
```

En este caso no basta con solo indicar el tipo, ya que es una colección, por lo tanto debo crear una colección de ese tipo.

****Se puede observar que si cambio el type de id a string, me mostraría errores en la colección.****

```
interface Audifono {
    id: number
    name: string
    image: string
    description: string
    price: number
}
```



TS: TYPES & INTERFACE

La recomendación es que si tengo un type que se deba reutilizar, mejor crear un archivo de types. Este se crea en su propia carpeta.

```
> EDITORES ABIERTOS
v AUDIFONOS-TS
  > node_modules
  > public
  v src
    > components
    > data
    v types
      TS types.ts

src > types > TS types.ts > [e] Audifono
1  export type Audifono = {
2      id: number
3      name: string
4      image: string
5      description: string
6      price: number
7  }
```

```
src > components > Audifono.tsx > ...
1  import type { Audifono } from "../types/types"
2
3  export function Audifono({ audifono } : { audifono : Audifono }) {
4
```



TS: TYPES & INTERFACE

Pero si no se reutiliza puedo agregarlo dónde se debe utilizar.

```
src > components > Audifono.tsx > Audifono
1  import type { Audifono } from "../types/types"
2
3  //esto es para un type que no se duplica
4  type AudifonoProps = {
5    audifono : Audifono
6  }
7
8  export function Audifono({audifono} : AudifonoProps) {
9
```